

第五章 管理 LVM 逻辑存储

LVM 把底层磁盘容量变成可分配、可扩展的逻辑块设备，但应用最终使用的仍然是挂载到目录树中的文件系统。做题时若把“磁盘、PV、VG、LV、文件系统、挂载点”混成一个对象，就容易出现局部命令成功、最终任务却没有完成的情况。本章按对象层级、典型操作和分层验收组织内容，所有命令都要回答一个明确问题。

[概念] 物理卷（Physical Volume, PV）、卷组（Volume Group, VG）与逻辑卷（Logical Volume, LV）构成 LVM 的三层容量对象。PV 把设备纳入 LVM；VG 汇聚一个或多个 PV，形成容量池；LV 再从 VG 中获得一段可供上层使用的逻辑块设备。

[概念] 物理区域（Physical Extent, PE）是 VG 的固定容量单位，逻辑区域（Logical Extent, LE）是 LV 的分配单位。在线性 LV 中，一个 LE 通常映射一个 PE，因此 $PE \text{ 大小} \times LE \text{ 数量}$ 可以得到 LV 的标称容量。

[概念] 文件系统与挂载点位于 LVM 之上。LV 已创建，只能证明块设备存在；创建 XFS 或 ext4 后，块设备才可以组织文件；完成挂载后，目录才真正指向该文件系统；写入 `/etc/fstab` 并验证后，挂载关系才具备启动持久性。

[操作语义] `pvs`、`vgs`、`lvs` 分别读取 PV、VG、LV 元数据；`pvcreeate`、`vgcreate`、`lvcreate` 则依次建立这三层对象。查询命令回答“现在是什么状态”，创建命令才改变对象。

[操作语义] `mkfs.xfs` 在块设备上新建 XFS，`mount` 建立当前挂载，`/etc/fstab` 描述持久挂载。

`lvextend --resizefs` 同时扩展 LV 与受支持的文件系统，因此验收时必须同时查看 LVM 层和文件系统层。

[知识专题] 对象层级、容量单位与查询边界

① [知识点] 六层对象不能互相替代

从一个空闲分区到应用可用目录，完整链路是：设备或分区进入 PV，PV 加入 VG，VG 划出 LV，LV 上创建文件系统，文件系统再挂载到目录。每一层都有独立状态，也有独立的查询入口。



这意味着以下结论都不能成立：设备存在不等于设备空闲；LV 存在不等于文件系统存在；目录存在不等于已经挂载；当前挂载成功不等于重启后仍会挂载。一个完整任务至少要跨越 LVM、文件系统与挂载三个层次进行验收。

② [知识点] PE、LE 与两种容量表达

PE 大小属于 VG。若 `projectvg` 的 PE 为 16 MiB，那么创建 50 个 LE 的线性 LV，其标称容量为 800 MiB：

$$50 \times 16 \text{ MiB} = 800 \text{ MiB}$$

`lvcreate` 和 `lvextend` 中，小写 `-l` 按 extent 数量或百分比表达，大写 `-L` 按容量表达。题目要求“50 个 extent”时应使用 `-l 50`；题目要求“800 MiB”时使用 `-L 800M`。参数并非两种随意替换的写法，而是在精确表达不同题意。

```
lvcreate -l 50 -n projectlv projectvg # 分配 50 个 extent
lvcreate -L 800M -n projectlv projectvg # 按容量分配 800 MiB
```

扩容命令还要区分目标总量和增量。`-l 70` 表示最终达到 70 个 extent，`-l +20` 表示在当前基础上增加 20 个 extent；加号不是装饰字符，而是改变操作语义的关键部分。

③ [操作点] 用一组查询建立对象映射

查询时最重要的参数是 `-o`，它让命令只显示本次判断需要的字段。`pvs` 重点看 PV 属于哪个 VG，`vgs` 重点看 PE 和空闲容量，`lvs` 重点看 LV 路径与底层设备；`findmnt` 与 `df` 则进入挂载和文件系统层。

<code>lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS</code>	# 块设备、文件系统与挂载点
<code>pvs -o pv_name,vg_name,pv_size,pv_free</code>	# PV 归属与剩余容量
<code>vgs -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count</code>	# VG、PE 与空闲 extent
<code>lvs -o lv_name,vg_name,lv_size,lv_path,devices</code>	# LV 路径、容量与底层映射
<code>findmnt /srv/project</code>	# 目录当前由哪个设备提供
<code>df -hT /srv/project</code>	# 文件系统类型、容量与使用率

`lvs` 与 `df` 的容量略有差异通常是正常的：前者显示块设备标称容量，后者显示文件系统视角的容量与可用空间。真正需要警惕的是扩容后 `lvs` 已明显增长，而 `df` 完全没有变化，这通常意味着只完成了 LV 层。

[Cheatsheet] 设备层看 `lsblk`；PV/VG/LV 分别看 `pvs`、`vgs`、`lvs`；目录挂载源看 `findmnt`；文件系统容量看 `df -hT`。先判断对象层级，再选择命令。

[操作专题] 从空闲设备建立 PV、VG 与 LV

① [操作点] 证明候选设备可以安全使用

`pvccreate` 会写入 LVM 标签和元数据，`mkfs` 会重建文件系统，因此任何写操作之前都必须重新确认设备身份。这里不需要把所有查询命令都执行一遍，但至少能回答：设备是否存在、是否已有文件系统、是否挂载、是否已经属于某个 VG。

<code>lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS</code>	# 确认设备结构与已有文件系统
<code>pvs -o pv_name,vg_name,pv_size,pv_free</code>	# 确认设备是否已经成为 PV

如果 `/dev/vdb1` 已出现文件系统类型、挂载点或现有 VG，操作应停止并重新核对题意。不能用强制选项绕过证据，也不能因为设备名“看起来像数据盘”就直接初始化。

② [操作点] 初始化 PV 并创建指定 PE 的 VG

这一阶段只需抓住 `vgcreate` 的一个重要参数：`-s`（physical extent size）设置 VG 的 PE 大小。它是 VG 属性，不属于 `pvccreate` 或 `lvcreate`。

<code>pvccreate /dev/vdb1</code>	# 把分区初始化为 PV
<code>vgcreate -s 16M projectvg /dev/vdb1</code>	# 创建 VG，并把 PE 设为 16 MiB

创建后立即检查成员关系和关键属性。`vg_extent_size` 应为 16 MiB，`vg_free_count` 表示尚可分配多少个 extent。

```
pvs -o pv_name,vg_name,pv_size,pv_free # /dev/vdb1 应属于 projectvg
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # 验证 PE 与余量
```

③ [操作点] 按题目要求创建 LV

`lvcreate` 中最常用的三个参数是：`-n` 指定 LV 名称，`-l` 按 extent 数量或百分比分配，`-L` 按容量分配。题目按 extent 描述时，不要先换算成近似容量再使用 `-L`。

```
lvcreate -l 50 -n projectlv projectvg # 创建 50 个 extent 的 projectlv
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices # 查询真实 LV 路径
```

若题目要求使用 VG 全部剩余空间，典型写法为 `-l 100%FREE`；只有题意明确要求占用全部剩余 extent 时才使用它，不能把它当成通用省事写法。

④ [验证点] 三层对象分别成立

PV、VG、LV 的创建结果应由各自查询命令证明。只看到最后一条 `lvcreate` 返回成功，不足以证明 PE 大小、设备归属和 LV 路径都符合要求。

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
```

[Cheatsheet] `pvcreeate <DEV>` 初始化 PV；`vgcreate -s <PE> <VG> <PV>` 创建指定 PE 的 VG；`lvcreate -l <LE> -n <LV> <VG>` 按 extent 创建 LV；按容量则把 `-l` 换成 `-L <SIZE>`。

[操作专题] 创建文件系统并建立持久挂载

① [操作点] 在新 LV 上创建 XFS

`mkfs.xfs` 的语义是“新建文件系统”，不是“让已有文件系统变大”。因此它只能作用于确认无需保留数据的新 LV。设备路径应优先来自 `lvs ... -o lv_path` 的查询结果。

```
mkfs.xfs /dev/projectvg/projectlv # 在新 LV 上建立 XFS
mkdir -p /srv/project # 创建挂载点目录
blkid /dev/projectvg/projectlv # 读取文件系统 UUID
```

若 LV 上已经存在需要保留的数据，重新执行 `mkfs.xfs` 会破坏原文件系统。扩容现有 XFS 时应使用 `lvextend --resizefs` 或 `xfs_growfs`，不能再次格式化。

② [操作点] 使用 UUID 写入 `/etc/fstab`

`/etc/fstab` 的六个字段依次是：设备标识、挂载点、文件系统类型、挂载选项、dump 标记、fsck 顺序。这里最重要的是：UUID 必须来自当前机器的 `blkid` 或 `lsblk -f`，不能复制示例值。

```
UUID=<真实 UUID> /srv/project xfs defaults 0 0
```

使用 UUID 的目的不是让配置显得更复杂，而是让挂载关系不依赖可能变化的 `/dev/...` 枚举顺序。

③ [操作点] 在重启前验证并执行挂载

`systemctl daemon-reload` 让 `systemd` 重新加载由 `fstab` 生成的 mount units；`findmnt --verify` 检查可解析性；`mount -a` 在当前会话中实际尝试挂载尚未挂载的条目。三者关注点不同，不能只执行其中一个就宣布完成。

```
systemctl daemon-reload      # 重新加载 fstab 对应的 mount units
findmnt --verify             # 检查 fstab 的明显语法与引用问题
mount -a                     # 在当前会话中实际尝试挂载
```

④ [验证点] 目录确实由目标 LV 提供

```
findmnt /srv/project          # 核对挂载源与文件系统类型
df -hT /srv/project           # 核对容量、类型与使用率
lsblk -f                      # 从块设备视角核对 UUID 和挂载点
```

目录存在没有证明力。若 `findmnt /srv/project` 没有结果，向该目录写入数据时，很可能实际写入根文件系统。

[Cheatsheet] 新 LV： `mkfs.xfs <LV_PATH>`；取 UUID： `blkid <LV_PATH>`；持久挂载：写 `fstab` 后执行 `daemon-reload`、`findmnt --verify`、`mount -a`；最终用 `findmnt` 和 `df -hT` 验收。

[知识专题] 扩容的容量来源、增量语义与文件系统边界

① [知识点] 扩容是一条容量链，不是一条命令

目录空间不足时，容量必须沿着 `PV → VG → LV → 文件系统 → 挂载点` 向上提供。VG 没有空闲 extent，LV 就无法继续增长；LV 已增长而文件系统未增长，目录也不会获得新增空间。因此扩容前至少要查看 `vg_free` 或 `vg_free_count`，扩容后至少要比 `lvs` 与 `df -hT`。

② [知识点] 目标值与增量值必须区分

`lvextend` 沿用 `-l` 和 `-L` 两套表达：小写按 extent，大写按容量；带 `+` 表示增加量，不带 `+` 表示目标总量。

<code>-l 70</code>	最终达到 70 个 extent
<code>-l +20</code>	在当前基础上增加 20 个 extent
<code>-L 1120M</code>	最终达到约 1120 MiB
<code>-L +320M</code>	在当前基础上增加约 320 MiB

VG 的 PE 为 16 MiB 时，增加 20 个 extent 等于增加 320 MiB。计算题的价值在于帮助检查参数是否表达了题目要求，而不是强迫把所有 extent 题都转换成 MiB。

③ [操作语义] `lvextend` 与 `xfs_growfs`

`lvextend --resizefs` 中，`--extents` 或 `-l` 指定 extent 目标，`--size` 或 `-L` 指定容量目标，`--resizefs` 或 `-r` 在扩大 LV 后继续调整受支持的文件系统。

```
lvextend --extents +20 --resizefs projectvg/projectlv # LV 与文件系统一起增加 20 PE
lvextend -L +320M -r projectvg/projectlv             # 按容量增量完成同一类操作
```

若已经单独扩大 LV，XFS 的文件系统扩容入口是已挂载目录：

```
xfs_growfs /srv/project # 扩展挂载在该目录的 XFS
```

XFS 支持增长，但不支持原地缩小。因此 `lvreduce` 不能被理解为 `lvextend` 的安全反向命令；先缩小 LV 会直接截断仍被文件系统使用的空间。

[Cheatsheet] 扩容前看 `vgs`；按 extent 用 `-l`，按容量用 `-L`；`+` 表示增量；优先使用 `lvextend -r` 同步扩展文件系统；XFS 可扩不可缩。

[操作专题] 在保留数据的前提下扩展 LV 与 XFS

① [操作点] 建立扩容前基线

先确认挂载点对应的 LV、文件系统类型和 VG 余量。`vg_free_count` 直接表示还剩多少个 extent，适合处理“增加 N 个 extent”的题目。

```
findmnt /srv/project # 挂载点与源设备
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path # 当前 LV 大小
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count # PE 大小与空闲 PE
df -hT /srv/project # 当前文件系统容量
```

需要证明数据被保留时，应在扩容前建立可复核的证据，而不是扩容后凭感觉判断。

```
printf 'project-data\n' > /srv/project/marker.txt # 建立需要保留的数据
sha256sum /srv/project/marker.txt # 记录扩容前校验值
```

② [操作点] VG 余量足够时直接扩展

这一条命令只需抓住两个参数：`--extents +20` 表示增加 20 个 extent，`--resizefs` 表示同步调整文件系统。

```
lvextend --extents +20 --resizefs projectvg/projectlv # 增加 LV，并让 XFS 使用新增空间
```

③ [操作点] VG 余量不足时先补充容量池

新设备不能直接成为 `lvextend` 的参数。它必须先经过 `pvccreate`，再通过 `vgextend` 加入目标 VG。
`vgextend` 的参数顺序是“目标 VG + 新 PV”。

```
pvccreate /dev/vdc1 # 把新分区初始化为 PV
vgextend projectvg /dev/vdc1 # 把 PV 加入现有 VG
vgs projectvg -o vg_name,vg_size,vg_free,vg_free_count # 重新确认空闲 PE
```

只有 `vg_free_count` 已满足要求后，才继续执行相同的 `lvextend`。反复重试空间不足的扩容命令不会自动发现新磁盘。

④ [验证点] LV、文件系统、挂载与数据同时正确

```
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path # LV 已增长
df -hT /srv/project # XFS 已使用新增空间
findmnt /srv/project # 挂载关系没有改变
sha256sum /srv/project/marker.txt # 原数据校验值保持一致
```

`lvs` 变大只能证明底层块设备增长；`df` 变大才说明文件系统使用了新增空间；校验值一致则证明任务没有通过重新格式化获得结果。

[Cheatsheet] 查余量：`vgs ... vg_free_count`；直接扩容：`lvextend -l +N -r <VG/LV>`；空间不足：`pvccreate <DEV> → vgextend <VG> <PV> → 再扩容`；验收：`lvs + df + findmnt + 数据校验`。

[诊断专题] 根据症状定位 LVM 失败所在层

① [诊断点] lvs 已增长，df 没变化

这说明 LV 层已经完成，但文件系统层仍停留在旧容量。先确认文件系统类型；XFS 可对已挂载目录执行 `xfs_growfs <MOUNT_POINT>`，也可以在扩容时直接使用 `lvextend -r`。继续重复 `lvextend` 只会让两层差距更大。

② [诊断点] 目录存在，业务仍写入根文件系统

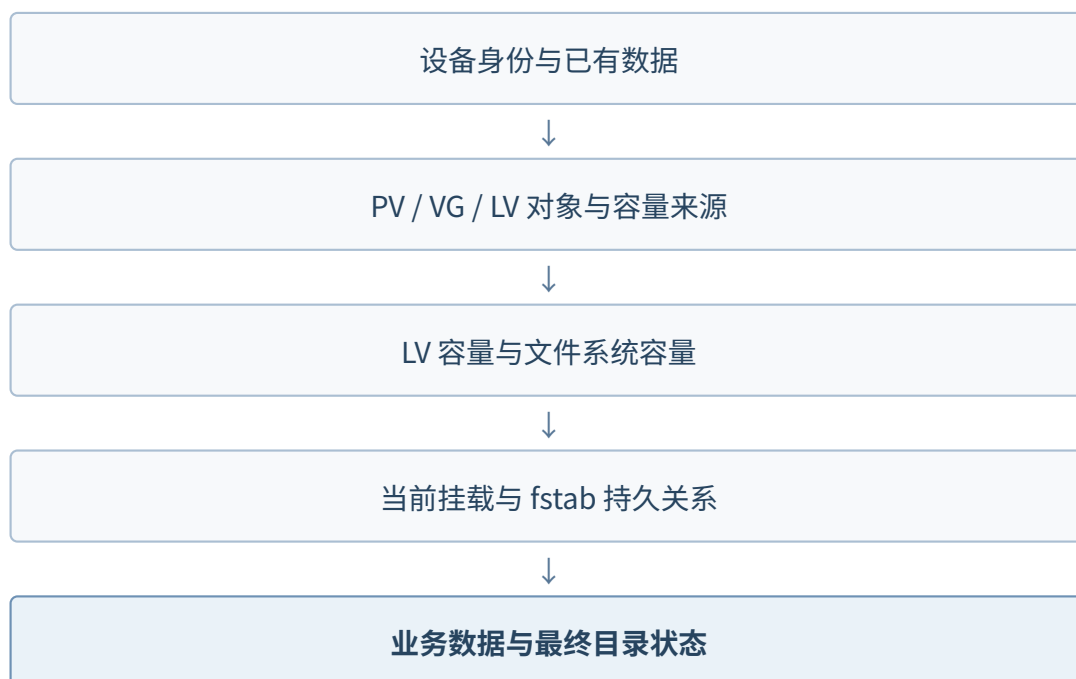
目录本身不代表已经挂载。使用 `findmnt <MOUNT_POINT>` 或 `mountpoint <MOUNT_POINT>` 判断当前挂载；再用 `findmnt --verify` 与 `mount -a` 检查持久配置。问题位于挂载层时，不应删除 LV 或重新格式化。

③ [诊断点] lvextend 报容量不足

查询 `vgs` 的 `vg_free` 与 `vg_free_count`。若 VG 余量不足，应寻找可安全使用的新设备，将其转换为 PV 并加入 VG；系统中“还有一块磁盘”并不等于目标 VG 已经拥有这块容量。

④ [诊断点] 候选设备已有文件系统或属于其他 VG

停止写操作，重新确认设备用途。`pvcreeate`、`mkfs`、`lvremove`、`vgremove`、`pvremove` 都是可能破坏数据的操作，不能作为“先清掉再重来”的通用排错路径。



[Cheatsheet] `lvs` 与 `df` 不一致查文件系统；目录存在但 `findmnt` 无结果查挂载；`lvextend` 空间不足查 VG 余量；设备已有签名或归属时停止破坏性操作。

[经典任务] 建立并扩展一个可持久挂载的项目卷

服务器提供两个确认空闲的分区 `/dev/vdb1` 与 `/dev/vdc1`。请完成以下目标：

1. 使用 `/dev/vdb1` 创建 `projectvg`，PE 大小为 16 MiB；
2. 创建 `projectlv`，初始大小为 50 个 extent；
3. 在 LV 上创建 XFS，并通过 UUID 持久挂载到 `/srv/project`；
4. 在 `/srv/project/marker.txt` 写入 `keep-this-data`；
5. 将 LV 再增加 20 个 extent，同时扩大文件系统并保留标记文件；
6. 若 VG 余量不足，先使用 `/dev/vdc1` 扩展 VG；
7. 不得重新格式化、删除原 LV 或缩减其他 LV。

验收时要能证明：PV 与 VG 关系正确，VG 的 PE 符合要求，LV 最终为 70 个 extent，XFS 已获得新增空间，挂载来源与 UUID 正确，标记文件仍然存在。

卡住时再想一想

`lvs` 变大，是否已经证明文件系统也变大？

新分区要经过哪两层，才能成为现有 LV 的可用容量？

目录存在，怎样证明它当前确实挂载了目标文件系统？

请先独立完成。参考分析与推荐实现从下一页开始。

[参考解答] 按对象依赖完成创建、挂载与扩容

① [操作点] 建立初始证据

先确认设备、已有 LVM 对象和挂载关系。这里的 `-o` 只保留后续决策真正需要的字段。

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS      # 设备、文件系统与挂载
pvs  -o pv_name,vg_name,pv_size,pv_free         # 现有 PV 与归属
vgs  -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # VG 余量与 PE
lvs  -o lv_name,vg_name,lv_size,lv_path,devices  # 现有 LV 与设备路径
```

② [操作点] 创建 PV、VG 与初始 LV

`-s 16M` 设置 VG 的 PE; `-l 50` 按 50 个 extent 分配; `-n projectlv` 设置 LV 名称。

```
pvcreeate /dev/vdb1                # 初始化 PV
vgcreate -s 16M projectvg /dev/vdb1 # 创建指定 PE 的 VG
lvcreate -l 50 -n projectlv projectvg # 创建 50 LE 的 LV
```

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
```

③ [操作点] 创建 XFS 并持久挂载

```
mkfs.xfs /dev/projectvg/projectlv # 新建 XFS
mkdir -p /srv/project              # 建立挂载点
blkid /dev/projectvg/projectlv     # 获取真实 UUID
```

将真实 UUID 写入 `/etc/fstab` :

```
UUID=<真实 UUID> /srv/project xfs defaults 0 0
```

```
systemctl daemon-reload # 重新加载 mount units
findmnt --verify         # 检查 fstab
mount -a                 # 当前会话实际挂载
findmnt /srv/project     # 验证挂载源
```

④ [操作点] 建立数据保留证据

```
printf 'keep-this-data\n' > /srv/project/marker.txt # 写入标记数据
sha256sum /srv/project/marker.txt                 # 记录校验值
```

⑤ [操作点] 解决容量来源并扩容

先看 VG 是否至少还有 20 个空闲 extent:

```
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若不足, 先让 `/dev/vdc1` 成为 PV, 再加入 `projectvg`:

```
pvccreate /dev/vdc1                                # 新设备进入 LVM
vgextend projectvg /dev/vdc1                       # 新 PV 加入容量池
vgs projectvg -o vg_name,vg_size,vg_free,vg_free_count # 再次确认余量
```

然后增加 20 个 extent, 并同步扩大 XFS:

```
lvextend --extents +20 --resizefs projectvg/projectlv # LV 与文件系统一起扩展
```

⑥ [验证点] 完成分层验收

```
pvs -o pv_name,vg_name,pv_size,pv_free            # PV 与 VG 关系
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # PE 与余量
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path # LV 最终容量
findmnt /srv/project                               # 当前挂载源
df -hT /srv/project                                # XFS 最终容量
blkid /dev/projectvg/projectlv                     # UUID
findmnt --verify                                     # 持久配置
sha256sum /srv/project/marker.txt                  # 数据未改变
```

VG 的 PE 应为 16 MiB; 70 个 extent 对应约 1120 MiB; `lvs` 和 `df` 都应体现增长; `findmnt` 应指向目标 LV; `fstab` 的 UUID 与 `blkid` 一致; 校验值保持不变。

[Cheatsheet] 调查 → `pvccreate` → `vgcreate -s` → `lvcreate -l/-L -n` → `mkfs` → UUID 写入 `fstab` → `findmnt --verify` + `mount -a` → 必要时 `pvccreate` + `vgextend` → `lvextend -l +N -r` → 分层验收。

[本章收束] 把命令还原为对象变化

LVM 的操作并不难，真正容易失分的是把不同层的成功混为一谈。`pvccreate`、`vgcreate`、`lvcreate` 分别建立三层容量对象；`mkfs`、`mount`、`fstab` 分别处理文件系统、当前挂载和持久关系；`lvextend -r` 同时推进 LV 与文件系统层。

创建题先确认设备身份，再沿对象依赖建立状态；扩容题先确认容量来源，再比较 LV 与文件系统是否同步增长；故障题则根据 `pvs`、`vgs`、`lvs`、`findmnt`、`df` 的证据判断失败位于哪一层。只要每条命令都能对应一个对象和一个验收点，LVM 就不会退化成需要死记的缩写清单。